xCurl 1.1.0

Manual prático de integração, fluxo e contratos

Passo a passo, sem pressupor conhecimento interno da biblioteca

Campo	Valor
Versão	1.1.0 estável
PHP suportado	^8.5.12
Data dos metadados da release	25/10/2026
Distribuição	Composer/Packagist recomendado; alternativas GitHub VCS e standalone
Repositório fonte	https://github.com/absolutoffice/xcurl
Documento	xCurl-1.1.0-Manual-practico-pt.pdf
Base de verificação	Código 1.1.0, metadados e regressões incluídas revistos em 25/10/2026

Para quem é este manual

Para utilizadores que precisam instalar, integrar, operar ou diagnosticar o xCurl sem deduzir o comportamento a partir de toda a implementação. Afirmações históricas de QA para mantenedores não são apresentadas como evidência atual da release.

Absolut Office - documentação do utilizador

0. Se só ler cinco minutos

O modelo completo do xCurl numa única página.

Regra mental principal

O xCurl tem duas camadas. Um TRANSPORTE executa exatamente um pedido HTTPS. O HttpClient decide redirects, retries, Retry-After e o deadline global. O HttpTransportFactory apenas seleciona um backend segundo as capacidades disponíveis no runtime.

Pergunta	Resposta curta
Como instalo?	Composer através do Packagist: composer require absolutoffice/xcurl:^1.1.
Que API devo usar normalmente?	HttpClient::send(HttpRequest, RequestOptions).
Como é selecionado o backend?	HttpTransportFactory::create() ou createStrict().
Há failover depois de uma falha do pedido?	Não. O fallback só acontece na seleção por capacidade do runtime.
O que é repetido por defeito?	GET/HEAD; falhas transitórias, 408, 429 e 5xx; no máximo dois retries.
Quem segue redirects?	Apenas HttpClient. O transporte devolve 3xx sem seguir.
Body null e vazio são iguais?	Não. null = sem entidade; string vazia = entidade explícita de 0 bytes.
Como descarrego para ficheiro?	RequestOptions(targetPath: ...). O target não pode existir.
Como diagnostico o backend?	HttpTransportFactory::diagnostics().
Como verifico o runtime instalado?	composer check-platform-reqs --no-dev, diagnostics() e um pedido HTTPS simples.

Mapa do manual

Caso	Motivo
1-4	Modelo mental, instalação, primeiro pedido e seleção de transporte.
5-8	Contratos de request/response, downloads e erros tipados.
9-13	Retries, redirects, tempos, TLS/limites e diagnóstico.
14-16	Verificação atual, distribuição Composer e mapa da API pública.
17-22	Receitas, antipadrões, troubleshooting, deployment, manutenção e referências.
Apêndice	Folha de consulta de uma página.

1. Modelo mental: o que cada camada faz

Decida primeiro se precisa de um pedido de baixo nível ou de uma operação lógica com política.

Conceito	O que significa no xCurl	O que NÃO significa
Fallback	Selecionar cURL, streams HTTPS ou socket conforme as capacidades atuais do runtime.	Tentar outro backend depois de uma falha DNS/TLS/leitura.
Retry	Repetir um pedido lógico segundo RetryPolicy usando o mesmo transporte selecionado.	Mudar automaticamente de transporte.
Redirect	HttpClient segue Location e aplica regras de método/body/credenciais.	O transporte seguir redirects sozinho.

Frase para recordar

Factory escolhe o motor. Transport faz uma tentativa. HttpClient organiza a operação.

Caso

Caso	Use	Motivo
Um pedido, sem política de retry/redirect	HttpTransportInterface::request()	Contrato pequeno e previsível.
Uso normal da aplicação	HttpClient::send()	Separa dados do pedido e política, mantendo o histórico.
Chamada escalar de compatibilidade	HttpClient::request()	Wrapper compatível sobre send().
Diagnóstico/teste determinista de backend	HttpTransportFactory::createNamed()	Não é a seleção normal de negócio.

2. Instalação e runtime

Composer é o caminho recomendado para consumidores em 1.1.0. O modo standalone continua disponível como fallback de compatibilidade.

Recomendado: Composer / Packagist

Instale a linha 1.1 no projeto consumidor. Composer aplica o requisito de PHP e fornece autoload PSR-4.

```
composer require absolutoffice/xcurl:^1.1
```

Alternativa direta por GitHub VCS

Use quando quiser fixar deliberadamente a tag estável do GitHub como repositório VCS em vez da resolução normal pelo Packagist.

```
composer config repositories.xcurl vcs https://github.com/absolutoffice/xcurl.git
composer require absolutoffice/xcurl:1.1.0 --no-plugins --no-scripts
```

Bootstrap da aplicação

Carregue uma única vez o autoloader do projeto consumidor. Não execute uma instalação Composer separada dentro de vendor/absolutoffice/xcurl.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
```

Nota de produção

O servidor de produção precisa da árvore vendor implementada e do autoloader gerado. O executável composer pode ficar no CI/build se as dependências forem instaladas antes do deployment.

Fallback standalone

O repositório mantém um autoloader local leve. Use-o apenas quando o deployment com Composer não for adequado; o loader standalone não executa os platform checks do Composer.

```
require_once '/path/to/xcurl/autoload.php';
```

3. Primeiro pedido: caminho recomendado

Use HttpClient quando a operação puder precisar de redirects, retries, deadline ou histórico de tentativas.

```
use AbsolutOffice\XCurl\Client\HttpClient;
use AbsolutOffice\XCurl\Client\RequestOptions;
use AbsolutOffice\XCurl\HttpRequest;
use AbsolutOffice\XCurl\HttpTransportFactory;

$client = new HttpClient(HttpTransportFactory::create());
$result = $client->send(
    new HttpRequest(
        'GET',
        'https://example.com/data.json',
        ['User-Agent: MyProject/1.0'],
    ),
    new RequestOptions(
        timeoutSeconds: 30,
        deadlineSeconds: 90,
    ),
);

$response = $result->response;
```

O que acontece

1. A factory verifica as capacidades do SAPI atual e cria um único transporte.
2. HttpClient recebe o request imutável e as opções de execução.
3. O transporte valida método, URL, headers, body, limites e target quando possível antes de I/O de rede.
4. O transporte selecionado executa um único pedido HTTPS.
5. HttpClient decide se devolve, redireciona ou repete.
6. O resultado inclui resposta final, URL final, contadores e histórico tipado.

Resposta HTTP != sucesso de negócio

Um 404, 409 ou 500 continua a ser uma resposta HTTP. O xCurl não lança exceção apenas por ser 4xx/5xx; a política pode repetir alguns status e a aplicação decide o significado final.

4. Seleção de transporte e fallback

A seleção depende das capacidades disponíveis, não de uma falha posterior de um pedido.

Modo	Ordem de seleção	Quando usar
create()	cURL -> streams HTTPS -> socket TLS	Compatibilidade geral; caminho padrão.
createStrict()	cURL -> socket TLS	Quando o xCurl deve aplicar verificação de framing mais estrita.

Não existe failover por pedido

Se create() selecionar cURL e o pedido falhar depois por DNS, timeout, TLS ou leitura, o xCurl não tenta silenciosamente streams ou socket. O HttpClient pode repetir o mesmo transporte quando a política permitir.

Sintoma

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

5. Contrato do request: entrada

Todos os transportes incluídos partilham a mesma validação para aceitar ou rejeitar a mesma entrada de forma consistente.

Campo	Regra	Exemplo
Método	Token HTTP válido; normalizado para maiúsculas; CONNECT é rejeitado.	get -> GET; CONNECT -> INVALID_REQUEST
HEAD	Um body explícito é rejeitado, incluindo string vazia.	HEAD + body "" -> INVALID_REQUEST
URL	Apenas HTTPS absoluta; URI ASCII RFC 3986; dados não ASCII devem vir percent-encoded; userinfo é rejeitado.	https://example.com/a%20b
Headers	Sem controlos inseguros; nome token; Host duplicado é rejeitado.	Host pode aparecer uma vez e deve ser autoridade válida.
Transfer-Encoding	Não suportado no request.	Use framing Content-Length exato.
Content-Length	Se fornecido, uma vez e igual aos bytes do body; caso contrário o xCurl sintetiza-o para body explícito.	"abc" -> 3 bytes

Semântica do body

Valor	Significado	Content-Length
null	Sem entidade de request.	Não sintetiza Content-Length; se o caller declarar, só pode ser 0.
""	Entidade explícita de zero bytes.	Sintetiza Content-Length: 0 se ausente.
"abc"	Entidade binária de 3 bytes.	Sintetiza Content-Length: 3 se ausente.

Limite do request

maxRequestBytes é opcional. Se excedido, falha antes da rede com TransportError::REQUEST_LIMIT não transitório. Um limite abaixo de 1 byte é inválido.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

6. Contrato da response: saída

Cada transporte incluído devolve um HttpResponse normalizado.

Campo	Significado
statusCode	Status HTTP final, 100..599.
headers	Bloco final normalizado; pode incluir trailers quando expostos.
body	Body em memória quando não é usado target file.
bytes	Bytes de body recebidos ou escritos.
targetCreated	true apenas quando essa tentativa concluída criou o target exclusivamente.

Semântica sem body

HEAD e status 204, 205 e 304 são respostas sem body. 101 Switching Protocols não é suportado. Uma resposta que termine apenas num 1xx informativo é erro de protocolo.

Framing estrito

Caso	Significado
Transfer-Encoding + Content-Length	Rejeitado por ambiguidade.
Transfer-Encoding diferente de chunked	Rejeitado.
Content-Length conflitante	Rejeitado.
Body menor que Content-Length	Erro READ transitório.
Streams HTTPS do PHP	Backend de compatibilidade; o wrapper pode ocultar Transfer-Encoding, por isso createStrict() exclui streams.

7. Downloads e limpeza segura por propriedade

A versão 1.1.0 torna explícita a propriedade do ficheiro. O target não é um destino para sobrescrever.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

- Com HttpClient, targetPath não pode existir; ficheiros e symlinks existentes são recusados antes do primeiro pedido.
- Os transportes incluídos criam targets de body exclusivamente. Respostas sem body não criam target.
- Numa falha de transporte, o transporte remove o parcial que criou.
- Antes de redirect ou retry por response, HttpClient remove apenas o ficheiro intermédio cujo response concluído indique targetCreated=true.
- Se aparecer um ficheiro e a tentativa concluída não reclamar propriedade, a continuação para com ClientError::TARGET_CLEANUP e o ficheiro não é apagado.
- Use um caminho temporário novo por operação; valide checksum/tamanho/regras de negócio e depois promova/renomeie na aplicação.

8. Erros: como perceber o que aconteceu

Não use mensagens de exceção para decidir lógica. Use enums, transient e histórico.

Erros do transporte

TransportError	value
DNS	dns
CONNECT	connect
TLS	tls
TIMEOUT	timeout
READ	read
WRITE	write
PROTOCOL	protocol
RESPONSE_LIMIT	response_limit
REQUEST_LIMIT	request_limit
INVALID_REQUEST	invalid_request
UNAVAILABLE	unavailable
UNKNOWN	unknown

Erros do cliente

ClientError	Significado
INVALID_REDIRECT	Destino de redirect múltiplo, inválido ou não HTTPS.
REDIRECT_LIMIT	Limite de redirects excedido.
DEADLINE	Tempo insuficiente para redirect, retry ou nova tentativa.
TARGET_CLEANUP	A continuação segura falhou por propriedade/limpeza do target intermédio.
TRANSPORT	Falha de transporte não repetida ou condições de retry esgotadas.

transient é decisivo

TransportError classifica a categoria. HttpTransportException::$transient indica se repetir o mesmo transporte pode razoavelmente resultar. RetryPolicy ainda verifica método e orçamento de retries.

```
try {
    $result = $client->send($request, $options);
} catch (AbsolutOffice\XCurl\Client\HttpClientException $e) {
    error_log('client_error=' . $e->error->value);
    foreach ($e->history as $attempt) {
        error_log($attempt->action->value);
    }
}
```

9. Retries: quando repete

A política padrão é conservadora para evitar duplicar efeitos laterais.

Parâmetro	Default	Efeito
maxRetries	2	No máximo duas repetições depois da primeira tentativa.
backoffMilliseconds	[1000, 3000]	Delay do retry #1 e #2; o último valor é reutilizado se necessário.
methods	[GET, HEAD]	Apenas estes métodos são retryable automaticamente por defeito.
statusCodes	[408, 429]	Status retryable explícitos.
retryServerErrors	true	Também repete 500..599.
maxRetryAfterSeconds	60	Limite aplicado a Retry-After válido.

POST não é repetido por defeito

Se ativar POST/PUT/PATCH em RetryPolicy, a idempotência é responsabilidade da aplicação/API. O xCurl não pode saber se repetir uma operação com efeitos laterais é seguro.

10. Redirects: regras exatas

HttpClient segue redirects manualmente; os transportes nunca ativam follow automático.

Status	Método seguinte	Body	Headers da entidade
301	preserva	preserva	preserva
302	preserva	preserva	preserva
303	GET exceto HEAD	remove se mudar método	remove Content-Length e Content-Type se mudar método
307	preserva	preserva	preserva
308	preserva	preserva	preserva

Regras de segurança dos headers

- Cada redirect seguido remove Host explícito para regenerar a autoridade.
- Redirects cross-origin removem Authorization por defeito.
- Redirects cross-origin removem Cookie por defeito.
- Ambos os comportamentos são configuráveis em RedirectPolicy.
- Múltiplos Location são inválidos enquanto o seguimento está ativo.
- maxRedirects: 0 significa não seguir; o 3xx é devolvido inalterado.
- A continuação continua a ser apenas HTTPS.

11. Timeout e deadline são diferentes

timeoutSeconds protege uma tentativa; deadlineSeconds limita toda a operação HttpClient.

Conceito	O que significa no xCurl	Efeito
timeoutSeconds	Uma tentativa de transporte.	Safety timeout com semântica específica do backend.
deadlineSeconds	Operação HttpClient completa.	Inclui retries, redirects e sleeps; limita cada nova tentativa.

Backend

Backend	Uso de timeoutSeconds
cURL	Timeout total do request e limite de conexão.
Streams HTTPS	Timeout nativo de I/O + verificação timed_out após leituras.
Socket TLS	Conexão TCP, I/O do handshake TLS e operações de stream posteriores.

Comportamento do deadline

Antes de cada nova tentativa o HttpClient calcula o tempo restante. Se restar menos de um segundo, ou se o delay mais outra tentativa não couber, falha com ClientError::DEADLINE.

12. TLS, CA privada e limites

A verificação TLS de peer e hostname permanece ativa. Um CA bundle personalizado altera as âncoras de confiança, não a política de verificação.

```
$caFile = __DIR__ . '/config/private-ca.pem';

$curl    = new CurlHttpTransport($caFile);
$streams = new StreamHttpTransport($caFile);
$socket  = new SocketHttpTransport($caFile);
```

Regra	Significado
Verificação de peer	Ativa.
Verificação de hostname	Ativa.
Caminho CA vazio/NUL	Configuração inválida.
maxResponseBytes < 1	Configuração inválida.
maxRequestBytes < 1	Configuração inválida.
Response excede limite	TransportError::RESPONSE_LIMIT, não transitório; parcial próprio removido.
Request excede limite	TransportError::REQUEST_LIMIT, não transitório; rejeitado antes da rede.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

13. Diagnóstico: o que cada SAPI pode usar

CLI e Apache/FPM podem usar a mesma versão de PHP com php.ini, extensões e capacidades diferentes. Verifique diagnostics() no SAPI que executa o pedido.

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Sintoma	Verificar
cURL=false	Verifique ext-curl e funções cURL necessárias.
streams=false	Verifique allow_url_fopen e wrapper https.
socket=false	Verifique ext-openssl e transports tls/ssl.
selectedStrict=socket com streams=true	Correto: strict exclui streams.

14. Verificação em 1.1.0

O checkout 1.1.0 tem uma superfície de verificação explícita e menor do que a árvore histórica 1.0. Não apresente relatórios históricos de 10/14 gates como evidência da 1.1.0.

Verificação do consumidor

Após instalar, valide a plataforma real, inspecione capacidades e execute um pedido HTTPS simples no SAPI alvo.

```
composer check-platform-reqs --no-dev
```

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Verificação do checkout

Na raiz do repositório, a documentação atual indica estes checks reproduzíveis:

```
composer validate --strict --no-check-lock
composer test
php tests/composer-consumer.php
find src tests -name '*.php' -print0 | xargs -0 -n1 php -l
php -l autoload.php
git diff --check
```

O que estes checks provam

- composer validate verifica metadados e consistência do pacote.
- composer test executa o runner de regressão sem dependências.
- tests/composer-consumer.php verifica rejeição do PHP 8.5.11, instalação simulada 8.5.12, smoke com vendor/autoload.php e conteúdo do archive Composer.
- Os checks de sintaxe cobrem src/tests/autoload.php atuais.

O que NÃO provam

- Não equivalem à suite histórica completa de integração cURL/streams/TLS.
- Não executam a antiga fire matrix nem o painel QA do browser; essas ferramentas não estão incluídas no checkout atual.
- Sozinhos não provam instalação remota a partir de GitHub ou Packagist.

15. Distribuição Composer e conteúdo do pacote

A versão 1.1.0 está estruturada como biblioteca Composer normal. O único requisito runtime do consumidor é PHP ^8.5.12; ferramentas de contribuição ficam em require-dev.

- PSR-4: AbsolutOffice\XCurl\ -> src/.
- Sem version no composer.json raiz; versões estáveis vêm de tags VCS.
- Archives Composer excluem índices locais, caches, vendor, ficheiros de ambiente, PDFs e material de chaves.
- LICENSE, COPYING, runtime e testes de regressão permanecem distribuídos segundo os checks atuais.
- Manuais PDF guardados no repositório ficam intencionalmente fora do pacote Composer pelas regras archive/export atuais.

```
"require": {
    "php": "^8.5.12"
},
"autoload": {
    "psr-4": {
        "AbsolutOffice\\XCurl\\": "src/"
    }
}
```

16. Mapa da API pública da linha 1.x

A referência API 1.0 continua como baseline histórico de compatibilidade para a linha 1.x. O código 1.1.0 mantém os tipos públicos centrais e acrescenta a indicação de propriedade HttpResponse::targetCreated.

Tipo	Papel	Recordar
HttpTransportFactory	Factory/diagnóstico	create, createStrict, createNamed, diagnostics, selected*
HttpTransportInterface	Contrato de transporte	request(...) -> HttpResponse
CurlHttpTransport	Transporte concreto	cURL
StreamHttpTransport	Transporte concreto	Streams HTTPS do PHP
SocketHttpTransport	Transporte concreto	Socket TLS direto
HttpRequest	Objeto de valor	method, url, headers, body
HttpResponse	Objeto de valor	statusCode, headers, body, bytes, targetCreated
HttpTransportException	Exceção de transporte	transient + TransportError
TransportError	Enum de transporte	12 categorias
Client\HttpClient	Orquestrador de política	send() preferido; request() wrapper
Client\RequestOptions	Opções de execução	timeoutSeconds, targetPath, deadlineSeconds
Client\RetryPolicy	Política de retry	methods, status, backoff, Retry-After
Client\RedirectPolicy	Política de redirect	limite + remoção de credenciais cross-origin
Client\HttpClientResult	Resultado final	response, finalUrl, history, redirects, retries
Client\HttpClientAttempt	Registo de tentativa	sequence, method, url, action, status/error/delay/nextUrl
Client\HttpClientAction	Enum do histórico	RETURNED, REDIRECTED, RETRIED, FAILED
Client\HttpClientException	Exceção do cliente	ClientError + history
Client\ClientError	Enum do cliente	5 categorias
ClockInterface	Contrato de extensão/teste	monotonicSeconds, unixTime
SleeperInterface	Contrato de extensão/teste	sleepMilliseconds

Superfície interna

Internal*, RequestValidator, SystemClock/SystemSleeper e helpers de capacidade são detalhes de implementação/teste; consumidores normais não devem construir lógica de negócio sobre eles.

17. Receitas copiáveis

Padrões pequenos para casos comuns.

A. GET simples

Use HttpClient + HttpRequest + RequestOptions e leia status/header/body.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
$response = $transport->request(
    'GET',
    'https://example.com/',
    [],
    30,
);

echo $response->statusCode, PHP_EOL;
```

B. POST JSON sem retry automático

POST não é retryable por defeito; passe body JSON raw e Content-Type.

```
$payload = json_encode($data, JSON_THROW_ON_ERROR);
$result = $client->send(
    new HttpRequest(
        'POST',
        'https://example.com/api/items',
        ['Content-Type: application/json'],
        $payload,
    ),
    new RequestOptions(timeoutSeconds: 30, deadlineSeconds: 60),
);
```

C. PUT binário com limites

Use createStrict(maxResponseBytes: ..., maxRequestBytes: ...) quando precisar de framing estrito.

```
$transport = HttpTransportFactory::createStrict(
    maxResponseBytes: 4 * 1024 * 1024,
    maxRequestBytes: 16 * 1024 * 1024,
);
```

D. Download grande para temporário

Use targetPath único; valide e promova após sucesso.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

E. Desativar redirects

Construa RedirectPolicy(maxRedirects: 0).

```
$client = new HttpClient(
    HttpTransportFactory::create(),
    redirectPolicy: new RedirectPolicy(maxRedirects: 0),
);
```

F. Retry de POST apenas quando a API for segura

Adicione POST a methods apenas com garantia real de idempotência.

```
$retry = new RetryPolicy(maxRetries: 1, methods: ['GET', 'HEAD', 'POST']);
$client = new HttpClient(HttpTransportFactory::create(), $retry);
```

18. Antipadrões

Evite estes erros de integração.

Não faça	Faça isto
“Se cURL falhar, tento streams manualmente.”	Mantenha o transporte selecionado; HttpClient só repete se a falha do mesmo transporte for transient e a policy permitir.
Colocar URLs/fornecedores/credenciais/regras de negócio no xCurl.	Mantenha-os na aplicação consumidora.
Ativar follow redirects no backend.	Deixe redirects para HttpClient.
Enviar Transfer-Encoding no request.	Envie body normal; o xCurl usa Content-Length exato.
Decidir lógica por texto de exceção.	Use TransportError/ClientError, transient e history.
Usar createNamed() como seleção normal.	Use create()/createStrict(); createNamed() é para diagnóstico/teste.
Reutilizar ou sobrescrever target de download.	Use um temporário novo por operação.
Assumir timeout=30 como exatamente 30.000 s em todos os backends.	Trate como safety timeout; use deadline para orçamento global.

19. Troubleshooting por sintomas

Comece pelo sintoma e diagnostics(); não mude transportes ao acaso.

Sintoma	Verificar	Motivo
“No usable HTTPS transport”	HttpTransportFactory::diagnostics()	Verifique cURL, allow_url_fopen/wrapper https e OpenSSL tls/ssl.
Funciona em CLI, falha no web	Compare diagnostics() nos dois SAPI	Verifique php.ini e extensões diferentes.
Falha TLS	TransportError::TLS + CA/hostname	Verifique SAN, cadeia e trust bundle.
Target recusado antes do request	O caminho já existe ou é symlink?	Use temporário novo.
Limpeza do target interrompe	ClientError::TARGET_CLEANUP	Apareceu ficheiro não próprio ou não pôde ser removido com segurança.
POST não repete	RetryPolicy methods	Default apenas GET/HEAD.
Redirect não seguido	RedirectPolicy maxRedirects + Location	0 desativa; Location deve resolver para HTTPS.
Deadline antes de maxRetries	deadlineSeconds + delays	Correto se não couber outra tentativa completa.
Streams aceita algo que strict rejeita	create() vs createStrict()	Correto: strict exclui streams.

20. Checklist de deployment do consumidor

O deployment normal da 1.1.0 é Composer-first.

1. Requerer absolutoffice/xcurl:^1.1 na aplicação consumidora.
2. Implementar a árvore vendor gerada pelo Composer.
3. Carregar vendor/autoload.php uma vez no bootstrap.
4. Escolher create() ou createStrict() conforme o framing.
5. Usar HttpClient::send() salvo necessidade deliberada de uma única chamada de transporte.
6. Definir timeout e, para operações multi-tentativa, deadline.
7. Definir limites de request/response quando necessário.
8. Manter credenciais, endpoints, checksums, schemas e regras de negócio fora do xCurl.
9. Para downloads, usar temporário novo e validar antes da promoção final.
10. Registar enums de erro e history, não depender do texto.
11. Opcionalmente executar composer check-platform-reqs --no-dev no runtime real.

Nota de produção

O servidor de produção precisa da árvore vendor implementada e do autoloader gerado. O executável composer pode ficar no CI/build se as dependências forem instaladas antes do deployment.

21. Compatibilidade e manutenção

xCurl 1.1.0 requer PHP ^8.5.12. A referência 1.0 permanece o baseline documentado da linha 1.x, enquanto 1.1.0 acrescenta mudanças compatíveis de distribuição e propriedade de downloads.

Alteração	Tratamento
Bugfix interno sem quebra pública	Pode entrar em patch/minor após checks atuais e revisão de compatibilidade.
Adição pública backward-compatible	Minor é adequada quando API/comportamento cresce de forma compatível.
Mudança que quebra assinaturas/semântica pública	Requer decisão explícita de versionamento semântico.
Mudança em helper interno/teste	Não é contrato do consumidor, mas regressões devem passar.
Referências históricas de QA	Manter como arquivo; não usar como evidência atual da 1.1.0.

22. Que fonte prevalece em caso de dúvida

Este manual explica. O código e os metadados atuais de release/pacote definem o comportamento do checkout 1.1.0.

Fonte	Use
src/	Implementação runtime exata.
composer.json	Nome, PHP, PSR-4, tooling dev e regras de archive.
README.md	Resumo operacional.
docs/INSTALLATION.md	Rotas de instalação e comandos atuais.
docs/releases/1.1.0.md	Mudanças e limites de validação da release.
tests/regression.php	Regressões incluídas sem dependências.
tests/composer-consumer.php	Checks de consumidor Composer e archive.
docs/reference/1.0.0/API-1.0.0.md	Referência histórica da API 1.0 para a linha 1.x.

Fecho

Mantenha as camadas separadas: a aplicação decide negócio; HttpClient decide policy; HttpTransportInterface faz um pedido; HttpTransportFactory seleciona o backend disponível.

Apêndice A. Folha de consulta

Preciso de...	Faça...
Instalar	composer require absolutoffice/xcurl:^1.1
Uso normal	new HttpClient(HttpTransportFactory::create()) + send(HttpRequest, RequestOptions)
Framing estrito	HttpTransportFactory::createStrict()
Um pedido sem policy	$transport->request(...)
Download	RequestOptions(targetPath: $tmp), onde $tmp não existe
Body raw	HttpRequest(..., body: $payload); null != ""
Limites	create(maxResponseBytes: ..., maxRequestBytes: ...)
Retries	RetryPolicy; default GET/HEAD, maxRetries=2
Redirects	RedirectPolicy; default maxRedirects=5
Orçamento total	RequestOptions(deadlineSeconds: ...)
Diagnosticar backend	HttpTransportFactory::diagnostics()
Falha de transporte	HttpTransportException: error + transient
Falha de cliente	HttpClientException: error + history
Verificar runtime	composer check-platform-reqs --no-dev
Regressão do checkout	composer test + php tests/composer-consumer.php

Ordem de seleção

Modo	1	2	3
create()	cURL	streams	socket
createStrict()	cURL	socket	-

Redirects

301	302	303	307	308
preserva	preserva	GET exceto HEAD; remove body se mudar o método	preserva	preserva